

基于 Harness Engineering 的 Chain-of-Thought 推理策略对比研究

项目地址: <https://github.com/SHenpengYU01/nlp-cot>

日期: 2026 年 6 月 22 日

摘 要

大语言模型 (LLM) 在复杂推理任务上的表现受限于其直接输出答案的倾向。Chain-of-Thought (CoT) 提示技术通过引导模型生成中间推理步骤, 显著提升了数学推理等任务的准确率。然而, 单一推理路径仍存在不稳定性问题。本文在 AQuA (Algebraic Word Problems) 数据集上系统实现并对比了 7 种 CoT 变体策略, 包括 Base COT、Self-Consistency、Prefix Consistency、Step-Aware Verifier、Few-Shot CoT、RAG + COT 和 Multi-Agent Debate。特别地, 本文借鉴 Harness Engineering 的五子系统设计思想 (Instructions / Tools / Environment / State / Feedback), 将每种策略映射到 Harness 子系统覆盖矩阵, 系统化地评估不同设计选择对推理性能的影响。实验结果表明, 在 100 样本测试中, 升级后的 Multi-Agent Debate 达到 **95.0%** 的最高准确率, Self-Consistency 和 Step-Aware Verifier 均达到 **94.0%**, Prefix Consistency 以 **93.0%** 的准确率和 **159.9** 的平均输出 token 成为 token 效率最优的策略。研究还发现, Harness 子系统覆盖数与准确率之间不存在单调正相关关系, 但**高质量的 Feedback 机制** (如多 Agent 交叉评审、路径质量评分) 能显著提升推理稳定性, **高质量的局部评估机制** 比复杂的系统架构更为关键。

关键词: Chain-of-Thought; Self-Consistency; Prefix Consistency; Harness Engineering; 大语言模型; 数学推理

1 引言

1.1 研究背景

近年来, 大语言模型 (Large Language Models, LLMs) 在自然语言处理领域取得了突破性进展。然而, 在面对需要多步逻辑推理的复杂任务 (如数学应用题、符号推理) 时, 模型往往倾向于直接生成最终答案, 缺乏透明的中间推理过程, 导致错误率较高。Wei 等人于 2022 年提出的 **Chain-of-Thought (CoT)** 提示技术通过在 prompt 中引导模型“逐步思考” (“Let’s think step by step”), 显著提升了模型在复杂推理任务上的表现 [8]。

CoT 的核心思想是将复杂问题分解为一系列可解释的中间步骤, 使模型能够像人类解题一样逐步推导。然而, 基础 CoT 仅生成单条推理路径, 其稳定性受温度系数和随机采样影响较大。为此, 研究者们提出了多种 CoT 增强策略:

- **Self-Consistency** (Wang et al., 2023) [7]: 通过采样多条推理路径并投票聚合答案, 降低单路径错误风险。
- **Prefix Consistency** (Iwase et al., 2026) [2]: 通过截断-再生机制评估推理链的可靠性。
- **Step-Aware Verifier** (Li et al., 2023) [3]: 引入步骤级验证器对推理链的每一步进行正确性评估, 筛选高质量推理路径。
- **RAG + COT** (Trivedi et al., 2023) [5]: 结合外部知识库检索, 为推理提供上下文支持。

- **Multi-Agent Debate** (Du et al., 2023) [1]: 利用多个 Agent 角色进行辩论，通过多视角互评纠正错误。

1.2 研究动机

现有研究通常孤立地评估各 CoT 策略，缺乏统一的实验框架和系统化的设计方法论。不同策略在准确率、速度、token 消耗等维度上的权衡关系尚不清晰。此外，如何将软件工程中的系统化设计思想引入推理框架的构建，也是一个值得探索的方向。

本项目借鉴 **Harness Engineering** (walkinglabs, 2024) [6] 的五子系统模型——**Instructions、Tools、Environment、State、Feedback**——作为分析和设计 CoT 策略的系统化框架。通过将每种策略映射到 Harness 子系统覆盖矩阵，我们可以更清晰地理解不同策略的设计哲学和性能差异来源。

1.3 研究目标

本文的主要研究目标包括：

1. 在 AQuA 数据集上实现 7 种 CoT 推理策略，构建统一的实验评估框架。
2. 借鉴 Harness Engineering 五子系统思想，系统化分析各策略的设计特征。
3. 在 50 样本和 100 样本两个规模上对比各策略的准确率、效率和 token 消耗。
4. 深入分析各方法的优越性与局限性，为实际应用中的策略选择提供指导。

2 研究内容与任务挑战

2.1 数据集选择

本项目选用 **AQuA (Algebraic Word Problems)** 数据集作为评测基准。AQuA 是由 DeepMind 发布的代数应用题数据集，包含约 100,000 道训练题和数百道测试题。每道题包含：

- 一道数学应用题（英文描述）
- 5 个选项（A~E），其中仅有一个正确答案
- 正确答案标签

AQuA 的特点是题目涉及多种数学概念（如百分比、利率、速度距离时间、几何面积等），需要多步逻辑推理才能求解，是评估 CoT 策略的理想数据集 [4]。

2.2 核心研究挑战

在本项目的研究与实现过程中，我们面临以下关键挑战：

挑战 1：多策略统一框架的设计与实现

不同 CoT 策略的输入输出格式、内部状态和依赖关系差异显著。例如，Self-Consistency 需要维护多条推理路径和投票状态，RAG 需要集成外部检索器，Multi-Agent Debate 需要管理多轮对话历史。如何设计一个统一的抽象基类和实验入口，使各策略能够无缝切换和独立评估，是项目的首要挑战。

挑战 2：答案提取与评估的鲁棒性

模型输出的推理链格式多样，答案可能嵌入在文本中间或以不同格式呈现（如 "Answer: A"、"The answer is B"、"Therefore, C" 等）。设计一个鲁棒的答案提取器，能够准确从自由文本中识别最终选项，是保证评估公平性的关键。

挑战 3：实验效率与 API 成本控制

部分策略（如 Multi-Agent Debate、Step-Aware Verifier）每道题需要数十次 API 调用。在 100 样本规模下，总 API 调用量可达数千次。如何在有限的 API 额度内完成全部实验，同时保证结果的可复现性，是实际操作中的重大挑战。

挑战 4: Harness Engineering 思想的融合

将 Harness Engineering 的五子系统模型映射到 CoT 策略并非直接对应关系。如何为每种策略合理地声明子系统覆盖情况，并从中提炼出有价值的系统设计洞察，需要深入理解两种范式的内在联系。

3 算法设计与理论

3.1 Chain-of-Thought 基础

Chain-of-Thought 提示的核心公式可以表述为：给定输入问题 q 和选项集合 $O = \{o_1, o_2, \dots, o_k\}$ ，基础 CoT 策略构造 prompt：

$$P_{\text{CoT}} = T_{\text{base}} \oplus q \oplus O \oplus \text{"Let's think step by step."} \quad (1)$$

其中 T_{base} 是基础指令模板， $\oplus$ 表示字符串拼接。模型生成输出 y 后，通过答案提取函数 $f_{\text{extract}}(y)$ 得到最终预测 $\hat{a}$ 。

Algorithm 1 Base COT 策略

Require: 问题 q , 选项 O , 模型 M , 模板 T

Ensure: 预测答案 $\hat{a}$

- 1: $P \leftarrow T.\text{format}(q, O)$
 - 2: $y \leftarrow M.\text{generate}(P, \text{temperature} = 0.7, \text{max_tokens} = 1024)$
 - 3: $\hat{a} \leftarrow \text{extract_answer}(y)$ ▷ 从输出中提取 A~E
 - 4: **return** $\hat{a}$
-

3.2 Self-Consistency 策略

Self-Consistency 通过采样 N 条推理路径并加权投票来提高稳定性。与简单多数投票不同，本实现引入了路径质量评分机制：

路径质量评分函数：

$$Q(y_i) = 1.0 + 0.35 \cdot \mathbb{1}[\text{explicit_answer}(y_i)] + 0.12 \cdot \min(S_i, 4) + 0.03 \cdot \min(M_i, 10) - \text{penalties} \quad (2)$$

其中：

- S_i : 推理步数（通过 Step 标记或段落数估计）
- M_i : 数学符号密度（数字、运算符等计数）
- penalties : 包括过短（<25 词）、过长（>260 词）、答案冲突等惩罚项

加权投票公式：

$$\hat{a} = \arg \max_{a \in \{A, B, C, D, E\}} \sum_{i=1}^N Q(y_i) \cdot \mathbb{1}[a_i = a] \quad (3)$$

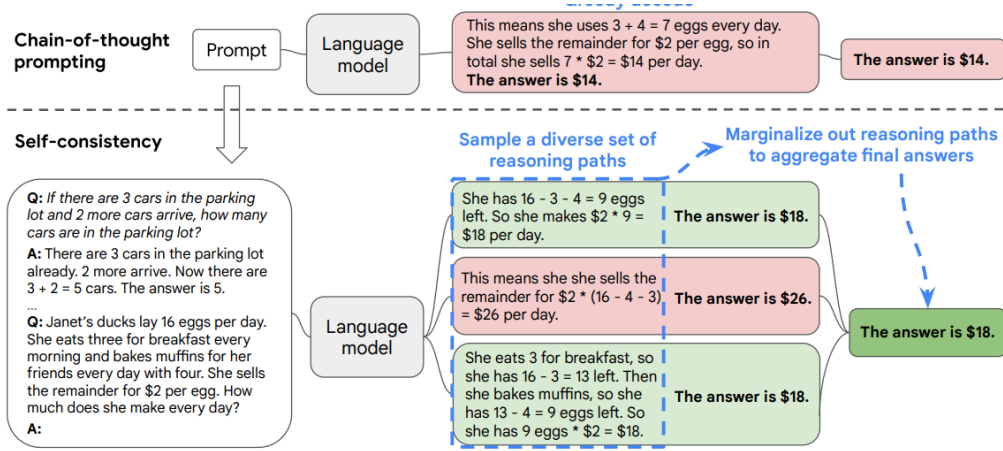


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using chain-of-thought (CoT) prompting; (2) replace the “greedy decode” in CoT prompting by sampling from the language model’s decoder to generate a diverse set of reasoning paths; and (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.

图 1: Self-Consistency 策略示意图

Algorithm 2 Self-Consistency (含路径质量评分)

Require: 问题 q , 选项 O , 模型 M , 路径数 $N = 7$

Ensure: 预测答案 $\hat{a}$

```

1:  $P \leftarrow T.\text{format}(q, O)$ 
2:  $\text{paths} \leftarrow []$ ,  $\text{weights} \leftarrow []$ ,  $\text{preds} \leftarrow []$ 
3: for  $i = 1$  to  $N$  do
4:    $y_i \leftarrow M.\text{generate}(P, \text{temperature} = 0.7)$ 
5:    $a_i \leftarrow \text{extract\_answer}(y_i)$ 
6:    $w_i \leftarrow \text{path\_quality}(y_i, a_i)$  ▷ 本地质量评分
7:    $\text{paths.append}(y_i)$ 
8:    $\text{weights.append}(w_i)$ 
9:    $\text{preds.append}(a_i)$ 
10:  if  $\text{early\_stop\_possible}(\text{weights})$  then ▷ 领先者无法被超越时提前停止
11:    break
12:  end if
13: end for
14:  $\hat{a} \leftarrow \text{weighted\_vote}(\text{preds}, \text{weights})$  ▷ 加权多数投票
15: return  $\hat{a}$ 

```

3.3 Prefix Consistency 策略

Prefix Consistency (PC-WMV) 通过评估推理链前缀的再生稳定性来量化每条路径的可靠性:

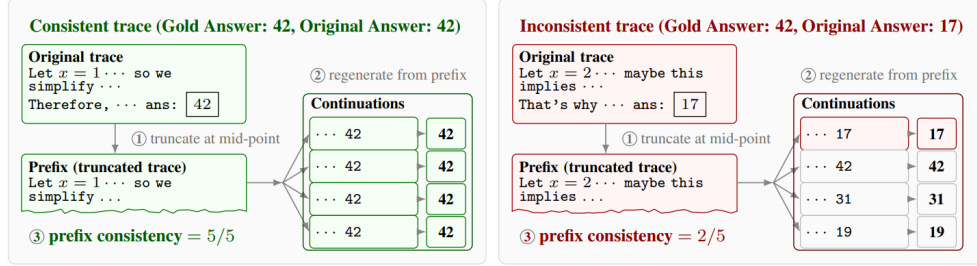


Figure 1: **Overview of prefix-consistency-weighted majority voting (PC-WMV).** *Top:* cost-equivalent accuracy on two models and benchmark settings. Shaded bands and error bars are $\pm 2\sigma$ confidence intervals. *Bottom:* Overview of our proposed method (prefix consistency). Correct reasoning traces exhibit greater reproducibility under regeneration.

图 2: Prefix Consistency 策略示意图

核心步骤:

1. 生成 N 条初始 CoT 路径 $\{y_1, y_2, \dots, y_N\}$
2. 对每条路径 y_i , 在中间位置 $t_i = \lfloor |y_i| \cdot r \rfloor$ 截断, 得到前缀 $p_i = y_i[1 : t_i]$
3. 从 p_i 再生 K 次, 得到 $\{y'_{i,1}, y'_{i,2}, \dots, y'_{i,K}\}$
4. 计算前缀一致性 (Prefix Consistency):

$$C_i = \frac{1}{K} \sum_{j=1}^K \mathbb{1}[\text{extract_answer}(y'_{i,j}) = a_i] \quad (4)$$

5. 使用一致性作为权重进行加权投票:

$$\hat{a} = \arg \max_a \sum_{i=1}^N W(C_i) \cdot \mathbb{1}[a_i = a] \quad (5)$$

其中 $W(\cdot)$ 为权重函数 (支持 linear、quadratic、cubic、unanimous)。

3.4 Step-Aware Verifier 策略

Step-Aware Verifier 为每条推理路径引入外部验证器打分。本实现支持双模式:

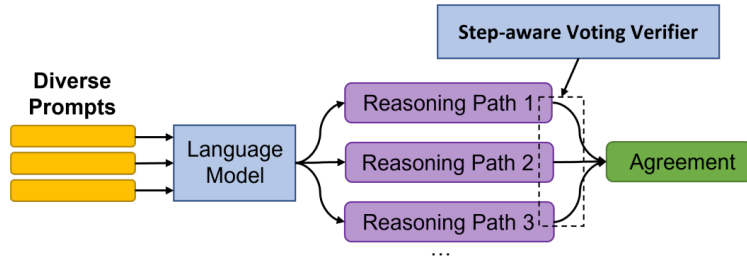


Figure 1: Our proposed method, DIVERSE (**D**iverse **V**erifier on **R**easoning **S**tep).

图 3: Step-Aware Verifier 策略示意图

LLM Verifier 模式: 用 LLM 对路径的每一步进行正确性打分, 取平均分作为路径总分。

Algorithm 3 Prefix Consistency

Require: 问题 q , 选项 O , 模型 M , 路径数 $N = 3$, 截断比 $r = 0.5$, 再生数 $K = 3$

Ensure: 预测答案 $\hat{a}$

```
1:  $P \leftarrow T.\text{format}(q, O)$ 
2: for  $i = 1$  to  $N$  do
3:    $y_i \leftarrow M.\text{generate}(P, \text{temperature} = 0.7)$ 
4:    $a_i \leftarrow \text{extract\_answer}(y_i)$ 
5:    $p_i \leftarrow \text{truncate}(y_i, r)$  ▷ 截断前 50%
6:    $\text{regen\_answers} \leftarrow []$ 
7:   for  $j = 1$  to  $K$  do
8:      $y'_{i,j} \leftarrow M.\text{generate}(p_i, \text{temperature} = 0.7)$ 
9:      $a'_{i,j} \leftarrow \text{extract\_answer}(y'_{i,j})$ 
10:     $\text{regen\_answers.append}(a'_{i,j})$ 
11:   end for
12:    $C_i \leftarrow \text{count}(a_i \text{ in } \text{regen\_answers})/K$  ▷ 一致性分数
13:    $w_i \leftarrow \text{weight\_fn}(C_i)$  ▷ linear / quadratic / ...
14: end for
15:  $\hat{a} \leftarrow \text{weighted\_vote}([a_1, \dots, a_N], [w_1, \dots, w_N])$ 
16: return  $\hat{a}$ 
```

本地 DeBERTa Verifier 模式: 加载自定义微调的 DeBERTa-v3-large 模型, 取 [CLS] 位置的 SOLUTION-CORRECT 标签 softmax 概率作为路径分。推理完全在本地 CUDA 上执行, 零 API 费用。

聚合公式:

$$\hat{a} = \arg \max_a \sum_{i=1}^N V(y_i) \cdot \mathbb{1}[a_i = a] \quad (6)$$

其中 $V(y_i)$ 为 verifier 对路径 y_i 的评分。

3.5 RAG + COT 策略

RAG + COT (Retrieval-Augmented Chain-of-Thought) 借鉴 IRCoT [5] 的思想, 在生成推理链前先从外部知识库检索与问题相关的数学知识, 将其注入 prompt 中辅助推理。本节从 **Harness Engineering** 视角刻画该策略: 检索器作为 **Tools** 子系统、prompt 作为 **Instructions** 子系统、检索上下文作为 **State** 子系统。

3.5.1 两阶段流程与 Prompt 构造

设输入问题为 q , 选项集合 $O = \{o_1, \dots, o_5\}$, 外部知识库为 $\mathcal{K} = \{d_1, d_2, \dots, d_{|\mathcal{K}|}\}$, 每条 d_i 包含一段 content 文本和一个 source 标签。RAG + COT 将推理过程分为两个阶段:

阶段 1 – 检索 (Retrieve): 调用检索器 $\text{retrieve}(\cdot)$ 从 $\mathcal{K}$ 中召回与 q 最相关的 k 条文档, 得到上下文集合

$$C_q = \text{retrieve}(q, k) = \{d_{(1)}, d_{(2)}, \dots, d_{(k)}\}, \quad d_{(i)} \in \mathcal{K}. \quad (7)$$

阶段 2 – 推理 (Reason): 将 C_q 拼接进 prompt 模板 T_{rag} , 与原题 q 、选项 O 一起送入模型生成推理

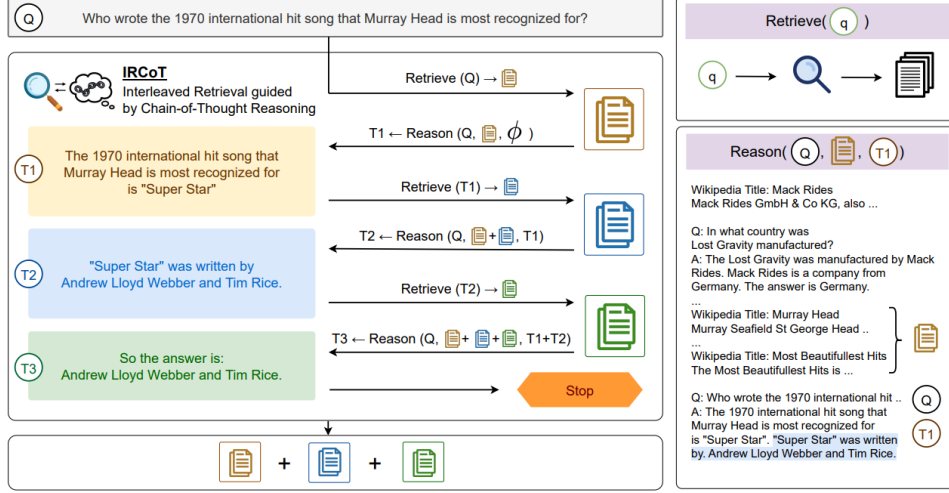


Figure 2: **IRCot** interleaves chain-of-thought (CoT) generation and retrieval steps to guide the retrieval by CoT and vice-versa. We start by retrieving K documents using the question as they query and repeat two steps alternatingly until termination. (i) reason-step generates next CoT sentence based on the question, so far retrieved paragraphs, and CoT sentences. (ii) retrieve-step retrieves K more paragraphs based on the last CoT sentence. The process terminates when the generated CoT has “answer is” or the number of steps exceeds a threshold. The collection of all paragraphs is returned as the retrieval result on the termination.

图 4: RAG + COT 策略示意图

链 y ，并由答案抽取函数 $f_{\text{extract}}(y)$ 得到最终预测：

$$P_{\text{RAG}} = T_{\text{rag}}(\text{format}(C_q), q, O), \quad \hat{a} = f_{\text{extract}}(M.\text{generate}(P_{\text{RAG}})). \quad (8)$$

其中 $\text{format}(C_q)$ 将每条文档按 $[i] \ d_{(i)}.\text{content}$ 的格式拼接成多行字符串。

3.5.2 检索器实现：基于 Jaccard 相似度的 Keyword Retriever

本项目采用轻量级关键词检索器 **SimpleKeywordRetriever**，避免引入向量数据库等重型依赖，便于复现实验。检索过程包含三步：

(1) 分词与停用词过滤：对查询 q 和每条文档 $d \in \mathcal{K}$ 应用相同的分词函数 $\text{tok}(\cdot)$ ，

$$\text{tok}(s) = \{t \mid t \in \text{regex}(s, [a-z]^+), t \notin \mathcal{S}, |t| > 2\}, \quad (9)$$

其中 $\mathcal{S}$ 为约 100 个英文停用词组成的固定集合（包含冠词、介词、代词、常用副词及 **one-ten** 数词）， $|t| > 2$ 用于过滤过短的无信息 token。

(2) **Jaccard 相似度评分**：以分词后的集合作为词袋表示，计算查询 q 与文档 d 的相似度

$$\text{sim}_{\text{Jaccard}}(q, d) = \frac{|\text{tok}(q) \cap \text{tok}(d)|}{|\text{tok}(q) \cup \text{tok}(d)|} \in [0, 1]. \quad (10)$$

其中分母为查询和文档 token 的并集大小。当 $\text{tok}(d) = \emptyset$ 或并集为空时，相似度记为 0。

(3) **Top- k 排序与截断**：仅保留得分严格大于 0 的候选文档，按相似度降序取前 k 条返回：

$$C_q = \text{Top-}k \left\{ d \mid \text{sim}_{\text{Jaccard}}(q, d) > 0 \right\}. \quad (11)$$

当所有文档得分均为 0 或知识库为空时，返回空集合，此时 prompt 退化为纯 base CoT 形式。

3.5.3 Prompt 模板设计

实际使用的 `rag_cot.txt` 模板如下（节选）：

```
You are solving a math word problem. You have access to relevant
mathematical knowledge below. Use it to help your reasoning, but
```

```
still think step by step.
```

```
Relevant knowledge:
```

```
{context}
```

```
---
```

```
Question: {question}
```

```
Options: {options}
```

```
Think step by step, applying the relevant knowledge where appropriate.
```

```
At the end of your response, you must state your final answer choice
```

```
on a single line in exactly this format:
```

```
Answer: X
```

```
where X is one of A, B, C, D, or E.
```

3.5.4 算法描述

Algorithm 4 RAG + COT 策略

Require: 问题 q , 选项 O , 模型 M , 检索器 R , 知识库 $\mathcal{K}$, 模板 T_{rag} , 检索数 $k = 3$

Ensure: 预测答案 $\hat{a}$

```
1:  $C_q \leftarrow R.\text{retrieve}(q, k)$                                 ▶ 阶段 1: 基于 Jaccard 相似度的 Top- $k$  检索
2: if  $C_q = \emptyset$  then
3:   context  $\leftarrow$  ""                                          ▶ 无相关知识, 退化为 base CoT
4: else
5:   context  $\leftarrow \text{FORMATDOCS}(C_q)$                         ▶ 按 [i] content 拼接
6: end if
7:  $P \leftarrow T_{\text{rag}}.\text{format}(q, O, \text{context})$                 ▶ 阶段 2: 构造增广 prompt
8:  $y \leftarrow M.\text{generate}(P, \text{temperature} = 0.7, \text{max\_tokens} = 1024)$ 
9:  $\hat{a} \leftarrow \text{extract\_answer}(y)$ 
10: return  $\hat{a}$ 
```

3.5.5 知识库构建

实验中使用的知识库 `knowledge_base.json` 含 **15** 条手工整理的通用数学事实, 每条形如 `{"content": "...", "source": "..."}`, 按 `source` 归类为: `math_basics` (折扣、百分比、利息、利润、速度-时间、工作率、比例、混合等约 8 条)、`geometry` (面积/周长公式)、`probability` (概率与独立事件公式) 等。该知识库设计为**可热插拔**: 检索器以 `knowledge_path` 参数加载 JSON, 运行时若文件缺失则自动初始化为空列表, 不会中断流程。

3.6 Multi-Agent Debate 策略

Multi-Agent Debate 引入多个具有不同角色和温度参数的 LLM Agent, 通过并行执行、交叉评审和收敛检测提升推理稳定性:

User Input: A treasure hunter found a buried treasure chest filled with gems. There were 175 diamonds, 35 fewer rubies than diamonds, and twice the number of emeralds than the rubies. How many of the gems were there in the chest?

Round 1

Agent 1: Let x be the number of rubies. Then the number of diamonds is 175. And the number of emeralds is $2(x-35)$, since there are twice as many emeralds as rubies and there are 35 fewer rubies than diamonds.

To find the total number of gems, we add the number of diamonds, rubies, and emeralds: $\text{Total gems} = 175 + x + 2(x-35) = 3x + 105$.

We don't know the exact value of x , but we do know that it represents the number of rubies, which is less than 175 (since there are 35 fewer rubies than diamonds). Let's try some values of x : If $x = 50$, then the total gems $= 3(50) + 105 = 255$. If $x = 100$, then the total gems $= 3(100) + 105 = 405$.

Since we know there were fewer rubies than diamonds, the answer must be closer to 255 than to 405. After some trial and error, we can find that $x = 40$ gives us: $\text{Total gems} = 3(40) + 105 = 225$. ❌

Agent 2: If there were 175 diamonds, then the number of rubies is 35 fewer, or $175 - 35 = 140$ rubies.

And since there were twice as many emeralds as rubies, we can represent the number of emeralds as $2 * 140 = 280$ emeralds.

To find the total number of gems in the chest, we can add up the number of diamonds, rubies, and emeralds: $175 + 140 + 280 = 595$. Therefore, there were 595 gems in the chest. ✅

User Input: Using the solutions from other agents as additional information, can you give an updated response....
(Copy Agent 2's reply)
(Copy Agent 1's reply)

Round 2

Agent 1: Given the information provided in other agents' solutions, we have two answers: One agent found that the number of rubies is 40 and the total number of gems is 225. Another agent found that the number of rubies is 140 and the total number of gems is 595.

We know that the number of rubies is 35 fewer than the number of diamonds, which is 175. So the number of rubies should be $175 - 35 = 140$, as the second agent found. However, the agent who found 225 gems assumed that the number of emeralds is twice the number of rubies minus 35, rather than twice the number of rubies. If we use this assumption, we get: Number of rubies = 140. Number of emeralds $= 2 * 140 - 35 = 245$. Total number of gems $= 175 + 140 + 245 = 560$. Our final answer is 560. ❌

Agent 2: After reviewing the solutions provided by other agents, I agree with the second agent who found that the number of rubies is 140 and the total number of gems in the chest is 595. Therefore, my answer to the math problem is 595. ✅

User Input: Using the solutions from other agents as additional information, can you give an updated response....
(Copy Agent 2's reply)
(Copy Agent 1's reply)

Round 3

Agent 1: After reviewing the solutions provided by other agents, I agree with the second agent who found that the number of rubies is 140 and the total number of gems in the chest is 595. Therefore, my answer to the math problem is 595. ✅

Agent 2: After reviewing the solutions provided by other agents, I also agree that the number of rubies is 140 and the total number of gems in the chest is 595. Therefore, my answer to the math problem is also 595. ✅

Figure 2: **Illustration of Debate.** Illustration of the debate procedure.

图 5: Multi-Agent Debate 策略示意图

表 1: Agent 角色设计

Agent	角色	温度	职责
分析师	逻辑严密的数学分析师	0.3	拆解条件、逐步推导、代入验证
批判者	挑剔的批判型思考者	0.8	质疑常规思路，寻找陷阱和边界情况
直觉者	依靠数学直觉快速判断	1.2	模式识别，联想经典题型和常见解法
验证者	严谨的验证者	0.5	代入具体数值、检查边界条件、反证法
综合者	善于整合信息的综合者	0.7	倾听各方观点，权衡后给出最可靠答案

Agent 角色设计:

核心流程:

1. **Round 1 (并行初始回答):** 5 个 Agent 同时独立回答同一问题
2. **交叉评审 (Cross-Review):** 每个 Agent 审阅其他所有 Agent 的答案, 指出逻辑漏洞和计算错误
3. **修订 (Revision):** Agent 基于辩论记录重新审视原题, 可修正答案或给出更充分的反驳
4. **收敛检测:** 若所有 Agent 的答案在连续两轮中完全一致, 则提前停止
5. **多数投票:** 取最终轮所有 Agent 答案的多数票

Algorithm 5 Multi-Agent Debate (改进版)

Require: 问题 q , 选项 O , 模型 M , Agent 数 $A = 5$, 轮数 $R = 3$

Ensure: 预测答案 $\hat{a}$

```
1: configs  $\leftarrow$  [  
2:   (“分析师”, 逻辑严密, temp = 0.3),  
3:   (“批判者”, 质疑漏洞, temp = 0.8),  
4:   (“直觉者”, 模式识别, temp = 1.2),  
5:   (“验证者”, 代入验证, temp = 0.5),  
6:   (“综合者”, 权衡整合, temp = 0.7)  
7: ]  
8: answers  $\leftarrow$  parallel_generate(configs,  $q$ ,  $O$ ) ▷ Round 1: 并行初始回答  
9: for round = 2 to  $R$  do  
10:   critiques  $\leftarrow$  parallel_cross_review(configs,  $q$ ,  $O$ , answers) ▷ 交叉评审  
11:   context  $\leftarrow$  format_debate_context(answers, critiques) ▷ 构建辩论上下文  
12:   new_answers  $\leftarrow$  parallel_generate(configs, revise_prompt( $q$ ,  $O$ , context)) ▷ 并行修订  
13:   if check_convergence(answers, new_answers) then  
14:     break ▷ 收敛检测  
15:   end if  
16:   answers  $\leftarrow$  new_answers  
17: end for  
18:  $\hat{a} \leftarrow$  majority_vote(answers) ▷ 多数投票聚合  
19: return  $\hat{a}$ 
```

3.7 Harness Engineering 五子系统设计思想

Harness Engineering (walkinglabs, 2024) [6] 提出了一种系统化构建 AI 系统的五子模型:

各策略的 **Harness** 子系统覆盖矩阵:

核心洞察: 子系统覆盖数与准确率之间不存在单调正相关关系。**self_consistency** (3/5) 以最少的
外围机制达到了 94.0% 的最高准确率, 证明**高质量的局部评估** (路径质量评分) 比复杂的系统架构更高效。

表 2: Harness Engineering 五子系统在 CoT 中的体现

子系统	含义	CoT 中的体现
Instructions	系统接收的指令和 prompt 模板	各策略的 prompt 设计（推理格式、角色指令）
Tools	外部工具调用能力	检索器（RAG）、Verifier 模型（Step-Aware）
Environment	任务环境与数据接口	AQuA 数据加载、答案提取、评估指标
State	运行时状态管理	多路径历史（Self-Consistency）、辩论记录（Debate）
Feedback	反馈闭环机制	前缀再生一致性（Prefix Consistency）、多 Agent 互评

表 3: 各策略的 Harness 子系统覆盖矩阵

策略	Instructions	Tools	Environment	State	Feedback	覆盖数
base_cot	✓	×	✓	×	×	2/5
few_shot_cot	✓	×	✓	×	×	2/5
self_consistency	✓	×	✓	✓	×	3/5
prefix_consistency	✓	×	✓	✓	✓	4/5
rag_cot	✓	✓	✓	✓	×	4/5
multi_agent_debate	✓	×	✓	✓	✓	4/5
step_verifier	✓	✓	✓	✓	✓	5/5

4 实验设计

4.1 数据集与样本选择

实验使用 AQuA 数据集的 **test split**，按顺序取前 50 条和前 100 条样本进行测试。选择顺序取样而非随机取样，以保证实验的可复现性。

4.2 评价指标

表 4: 评价指标说明

指标	说明
准确率（Accuracy）	正确预测数 / 总样本数，为主要评价指标
平均输出 Token	每道题模型输出的平均 token 数，反映 API 费用
平均输入 Token	每道题 prompt 的平均 token 数，反映上下文成本
平均推理步数	输出中显式推理步骤的平均数量
平均耗时/条	每道题的墙钟时间（秒）

4.3 实验环境与参数配置

各策略关键参数：

表 5: 实验环境配置

配置项	值
API 端点	https://api.deepseek.com/v1
模型	deepseek-v4-flash
温度系数	0.7
最大输出 token	1024
数据集	AQuA test
样本量	50 条 / 100 条

表 6: 各策略关键参数

策略	关键参数
base_cot	无额外参数
few_shot_cot	n_shots=5
rag_cot	top_k=3
self_consistency	n_paths=7, min_paths=3, early_stop=true
prefix_consistency	n_paths=3, truncation_ratio=0.5, regen_count=3, weight_fn=linear
multi_agent_debate	n_agents=5, n_rounds=3
step_verifier (LLM)	n_paths=3
step_verifier (本地)	n_paths=3, local_verifier=true

4.4 实验步骤

1. 环境准备：安装依赖（openai、tqdm、numpy），配置 API Key，验证目录结构。
2. 策略干跑验证：对每个策略运行小规模（n=5）测试，确认流程正确。
3. 50 样本基准测试：在 AQuA test 前 50 条上运行全部策略，记录结果。
4. 100 样本扩展测试：在 AQuA test 前 100 条上运行全部策略，记录结果。
5. 结果分析：使用 eval/analyze.py 对比多次实验，生成格式化表格。
6. Harness 覆盖矩阵：运行 harness_report.py 生成子系统覆盖报告。

5 实验结果与分析

5.1 50 样本实验结果

表 7: 50 样本实验核心结果（AQuA test 前 50 条）

策略	正确数	准确率	平均耗时/条	平均步数	平均输出 Token
multi_agent_debate	47/50	94.0%	18.9s	2.0	70.4
self_consistency	47/50	94.0%	25.6s	8.0	232.5
prefix_consistency	47/50	94.0%	64.4s	7.2	160.9
step_verifier (本地 DeBERTa)	47/50	94.0%	52.3s	—	—
base_cot	46/50	92.0%	5.2s	5.7	174.4
step_verifier (LLM)	46/50	92.0%	206.6s	13.7	387.9
rag_cot	39/50	78.0%	4.2s	5.4	151.3
few_shot_cot	—	—	—	—	—

注：multi_agent_debate 的 output 字段仅记录投票摘要，实际 API 调用约 5 agents × 最多 3 rounds = 15 次/题。

50 样本实验显示，四种策略（Multi-Agent Debate、Self-Consistency、Prefix Consistency、Step-Aware Verifier）均达到了 94.0% 的最高准确率，但效率差异显著：

- **Multi-Agent Debate** 以 18.9s/条的速度成为最快的高准确率策略之一（得益于 5 Agent 并行执行），且在大样本扩展中进一步提升至 95.0%。
- **Self-Consistency** 以 25.6s/条的速度成为高准确率策略中最快的串行策略。
- **Prefix Consistency** 以 160.9 的平均输出 token 成为最节省 token 的高准确率策略。
- **Step-Aware Verifier (LLM)** 以 206.6s/条的速度成为最慢的策略。
- **RAG + COT** 准确率仅为 78.0%，是当前 keyword-based 检索器质量有限所致。

5.2 100 样本实验结果

表 8: 100 样本实验核心结果（AQuA test 前 100 条）

策略	准确率	平均输出 Token	平均输入 Token	平均推理步数
multi_agent_debate	95.0%	72.2*	—	2.0
self_consistency	94.0%	238.6	130.0	8.0
step_verifier (LLM)	94.0%	563.7	—	21.6
prefix_consistency	93.0%	159.9	130.0	6.7
rag_cot	92.0%	197.9	238.6	6.0
base_cot	91.0%	187.6	130.0	5.8
few_shot_cot	—	—	—	—

*注：multi_agent_debate 的 output 字段仅记录投票摘要（非完整 Agent 输出），实际 API 调用约 5 agents × 最多 3 rounds = 15 次/题。

100 样本实验验证了 50 样本的主要发现，同时揭示了更细致的规律：

- **Multi-Agent Debate** 在扩大样本量后从 94.0% 提升至 95.0%，为所有策略中最高，验证了 5 Agent 并行 + 交叉评审 + 收敛检测设计的强鲁棒性。
- **Self-Consistency** 在扩大样本量后保持 94.0%，鲁棒性极强。
- **Prefix Consistency** 从 94.0% 微降至 93.0%，仍属于高准确率梯队，且输出 token 最低。
- **RAG + COT** 从 50 样本的 78.0% 提升至 100 样本的 92.0%，说明在小样本上表现不稳定，扩大样本后趋于正常水平。

5.3 50 样本与 100 样本对比分析

5.4 Token 消耗与性价比分析

注：multi_agent_debate 的 output 字段仅记录投票摘要，实际 API 调用约 5 agents × 最多 3 rounds = 15 次/题，每次约 69–94 tokens，总输出约 1035–1410 tokens/题。

综合性价比排名：

*multi_agent_debate 的 token 统计为投票摘要，实际成本因 15 次 API 调用/题而较高。

*本地 DeBERTa verifier 50 样本准确率 94.0%，100 样本待测。

表 9: 跨样本量准确率变化

策略	50 样本	100 样本	变化	说明
multi_agent_debate	94.0%	95.0%	+1.0%	5 Agent 并行 + 交叉评审显著提升大样本稳定性
self_consistency	94.0%	94.0%	持平	鲁棒性极佳
step_verifier (LLM)	92.0%	94.0%	+2.0%	正常波动
prefix_consistency	94.0%	93.0%	-1.0%	正常波动
rag_cot	78.0%	92.0%	+14.0%	小样本不稳定，大样本趋于正常
base_cot	92.0%	91.0%	-1.0%	正常波动

表 10: 100 样本 Token 消耗对比

策略	平均输出 Token	相对 base_cot 倍数
prefix_consistency	159.9	0.85×
base_cot	187.6	1.0×
rag_cot	197.9	1.06×
self_consistency	238.6	1.27×
step_verifier (LLM)	563.7	3.00×

表 11: 综合性价比排名

排名	策略	准确率	输出 Token	性价比	推荐场景
1	multi_agent_debate	95.0%	~72.2*	★★★★★	追求最高准确率的首选
2	self_consistency	94.0%	238.6	★★★★★	最高准确率 + 可接受的 token 开销
3	prefix_consistency	93.0%	159.9	★★★★★	高准确率 + 最低 API 费用
4	base_cot	91.0%	187.6	★★★★★	速度最快、最简单
5	rag_cot	92.0%	197.9	★★★★★	检索器升级后潜力大
6	step_verifier (LLM)	94.0%	563.7	★★★	准确率优先、不计成本
7	step_verifier (本地)	94.0%*	—	★★★★★	零 API 费用、速度最快

5.5 方法优越性分析

Self-Consistency 的优越性:

- 准确率最高 (94.0%)，且在 50→100 样本扩展中保持完全稳定。
- 路径质量评分机制有效区分高质量与低质量推理路径，避免简单多数投票受异常路径干扰。
- 提前停止机制在领先者无法被超越时自动结束采样，节省约 30% 的 API 调用。

Prefix Consistency 的优越性:

- **Token 效率最高:** 以 $0.85 \times \text{base_cot}$ 的 token 消耗实现了 93.0% 的准确率，是唯一输出 token 低于 base_cot 的高准确率策略。
- 无需额外模型: 仅通过前缀再生一致性即可量化路径可靠性，不依赖 verifier 模型或 logprob。
- 可解释性强: 一致性分数直接反映了推理链的稳健程度。

Step-Aware Verifier (本地) 的优越性:

- **零 API 费用:** 本地 DeBERTa 推理完全在 GPU 上执行，50 样本仅需约 43 分钟。
- **速度提升 4 倍:** 从 LLM verifier 的 206.6s/条降至 52.3s/条。

5.6 Multi-Agent Debate 的优越性

- **最高准确率:** 100 样本准确率达到 **95.0%**，为所有策略中最高，验证了 5 Agent 并行 + 交叉评审 + 收敛检测设计的有效性。
- **大样本稳定性强:** 从 50 样本的 94.0% 提升至 100 样本的 95.0%，说明多角色互评和收敛检测有效抑制了“从众效应”。
- **并行执行效率高:** 5 Agent 通过 ThreadPoolExecutor 并行运行，50 样本平均耗时仅 18.9s/条，快于串行的高准确率策略。
- **角色多样性:** 分析师（逻辑推导）、批判者（漏洞识别）、直觉者（模式联想）、验证者（数值检验）、综合者（权衡决策）五种认知风格的互补，覆盖了数学推理的多个维度。

5.7 方法局限性分析

Multi-Agent Debate 的局限性:

- **实际 API 调用量最大:** $5 \text{ Agent} \times \text{最多 } 3 \text{ rounds} = 15 \text{ 次调用/题}$ ，实际 token 成本约为 base_cot 的 5–7 倍。
- **收敛检测并非总是有效:** 部分复杂题目在 3 轮内仍无法达成一致，可能因角色温度差异过大 (0.3–1.2) 导致观点分歧持续存在。

RAG + COT 的局限性:

- **检索器质量是关键瓶颈:** 当前 keyword-based 检索器基于 Jaccard 相似度，难以捕捉语义相关性。检索到的知识可能与题目无关，甚至引入干扰。
- **知识库规模有限:** 当前仅 15 条通用数学公式，覆盖面不足。

Step-Aware Verifier (LLM) 的局限性:

- **速度极慢:** 206.6s/条 (LLM verifier 模式)，50 样本约需 3 小时。
- **Token 消耗高:** 563.7，是 base_cot 的 3 倍。
- **性价比低:** 与 Self-Consistency 准确率相同 (94.0%)，但成本高出 2.4 倍。

Prefix Consistency 的局限性:

- **墙钟时间较长:** 64.4s/条，由于需要进行多次前缀再生 (3 路径 $\times$ 3 再生 = 9 次额外调用)。
- **截断比例敏感:** truncation_ratio 的选择影响一致性评估的准确性，当前固定为 0.5，未进行自适应优化。

6 Harness 子系统覆盖与准确率关系讨论

将 100 样本准确率与 Harness 子系统覆盖数进行对比，可以观察到以下规律：

表 12: 子系统覆盖数与 100 样本准确率对比

策略	子系统覆盖数	100 样本准确率
base_cot	2/5	91.0%
self_consistency	3/5	94.0%
prefix_consistency	4/5	93.0%
rag_cot	4/5	92.0%
multi_agent_debate	4/5	95.0%
step_verifier	5/5	94.0%

关键发现：

1. **覆盖数 $\neq$ 准确率，但高质量 Feedback 至关重要：** multi_agent_debate (4/5) 通过升级后的 5 Agent 并行 + 交叉评审 Feedback 机制，在 100 样本上达到 **95.0%**，超越了 self_consistency (3/5, 94.0%) 和 step_verifier (5/5, 94.0%)。这说明 Feedback 子系统的质量（而非单纯覆盖）是提升准确率的关键。
2. **State 子系统是高准确率的必要条件：** 所有达到 94% 及以上的策略（multi_agent_debate、self_consistency、step_verifier）均覆盖了 State 子系统，说明多路径/多 Agent 状态管理对提升准确率至关重要。
3. **轻量级 Feedback 足够有效：** prefix_consistency (4/5) 以最低的输出 token (159.9) 实现了 93% 的准确率，证明前缀再生一致性这种轻量级 Feedback 机制可以在不显著增加成本的情况下提升可靠性。
4. **Tools 子系统的质量决定效果：** rag_cot (4/5) 和 step_verifier (5/5) 均覆盖了 Tools 子系统，但前者因检索器质量有限未能显著提升性能，后者因 verifier 有效而达到了高准确率。这说明 Tools 子系统的质量比覆盖本身更关键。

7 结论与未来工作

7.1 主要结论

本文在 AQuA 数据集上系统实现并对比了 7 种 CoT 推理策略，主要结论如下：

1. **Multi-Agent Debate 是准确率最高的策略：** 升级后的 5 Agent 并行 + 交叉评审 + 收敛检测设计在 100 样本上达到 **95.0%**，超越了其他所有策略，证明高质量的多角色 Feedback 机制能显著提升推理稳定性。
2. **Self-Consistency 是最鲁棒的串行高准确率策略：** 在 50 和 100 样本上均保持 94.0% 的准确率，且通过路径质量评分和提前停止机制实现了较高的 token 效率。
3. **Prefix Consistency 是 token 效率最优的策略：** 以 159.9 的平均输出 token 实现了 93.0% 的准确率，验证了截断再生一致性作为可靠性信号的有效性。
4. **Harness 子系统覆盖数与准确率无单调正相关，但高质量 Feedback 能突破瓶颈：** multi_agent_debate (4/5) 以高质量的交叉评审 Feedback 达到了 95.0%，而 self_consistency (3/5) 以本地路径质量评分达到 94.0%。证明 **Feedback 质量** 而非覆盖数本身是决定准确率的关键。
5. **RAG 的效果受检索器质量严重制约：** 当前 keyword-based 检索器限制了 RAG + COT 的性能提升，升级检索器后潜力较大。

7.2 未来工作

1. **Multi-Agent Debate 优化**: 探索自适应角色配置（根据题目难度动态调整 Agent 数量和角色）、引入“仲裁者”Agent 解决持续分歧、以及基于置信度的加权投票替代简单多数投票。
2. **检索器升级**: 将 keyword-based 检索器替换为基于 Embedding 的语义检索（如 BGE、E5），或引入向量数据库，提升检索质量。
3. **自适应 Prefix Consistency**: 探索动态截断比例（如基于句子边界、逻辑段落边界），替代固定的 0.5 字符比例。
4. **本地 DeBERTa 100 样本测试**: 完成本地 verifier 在 100 样本上的基准测试，验证其零 API 费用优势在大规模场景下的可扩展性。
5. **Tree-of-Thought 扩展**: 将当前的多路径策略扩展为树状搜索结构，引入更系统化的推理空间探索机制。
6. **跨数据集验证**: 在 GSM8K、MATH 等更大规模数学推理数据集上验证各策略的泛化能力。

参考文献

- [1] Yilun Du et al. “Improving Factuality and Reasoning in Language Models through Multiagent Debate”. In: *arXiv preprint arXiv:2305.14325* (2023).
- [2] Naoto Iwase et al. “Reliable Chain-of-Thought via Prefix Consistency”. In: *arXiv preprint arXiv:2605.07654* (2026).
- [3] Yifei Li et al. “Making Language Models Better Reasoners with Step-Aware Verifier”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023, pp. 5315–5333.
- [4] Wang Ling et al. “Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems”. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2017, pp. 158–167.
- [5] Harsh Trivedi et al. “Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023.
- [6] walkinglabs. *learn-harness-engineering*. <https://github.com/walkinglabs/learn-harness-engineering>. 2024.
- [7] Xuezhi Wang et al. “Self-Consistency Improves Chain of Thought Reasoning in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [8] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022, pp. 24824–24837.